

```
1 package com.tp.jpa.model.entities;
2 import jakarta.persistence.GeneratedValue;
3 import jakarta.persistence.GenerationType;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.MappedSuperclass;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8 import java.time.LocalDateTime;
9
10 @MappedSuperclass
11 @Getter
12 @Setter
13 @ToString
14 @SuperBuilder
15 @AllArgsConstructor
16 @NoArgsConstructor
17 @EqualsAndHashCode(of = {"id"})
18 public abstract class Base {
19
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.
        IDENTITY)
23     private Long id;
24     @Builder.Default
25     protected boolean eliminado = false;
26     @Builder.Default
27     protected LocalDateTime createdAt = LocalDateTime
        .now();
28 }
```

```
1 package com.tp.jpa.model.entities;
2
3 import ar.edu.tup.programacion3.enums.Estado;
4 import ar.edu.tup.programacion3.enums.FormaPago;
5 import jakarta.persistence.*;
6 import lombok.*;
7 import lombok.experimental.SuperBuilder;
8
9 import java.time.LocalDate;
10 import java.util.HashSet;
11 import java.util.Set;
12 @Entity
13 @Table(name = "pedidos")
14 @Getter
15 @Setter
16 @ToString
17 @EqualsAndHashCode(callSuper = true)
18 @AllArgsConstructor
19 @NoArgsConstructor
20 @SuperBuilder
21 public class Pedido extends Base implements
    Calculable {
22     private LocalDate fecha;
23     @Enumerated(EnumType.STRING)
24     private Estado estado;
25     private Double total;
26     @Enumerated(EnumType.STRING)
27     private FormaPago formapago;
28     @ManyToOne
29     @JoinColumn(name = "usuario_id")
30     private Usuario usuario;
31
32     //coleccion de detalle pedidos
33     @Builder.Default
34     @OneToMany(mappedBy = "pedido", cascade =
        CascadeType.ALL, orphanRemoval = true)
35     private Set<DetallePedido> detallePedidos = new
        HashSet<>();
36
37
38     //Agregar detalle
```

```
39     public void addDetallePedido(int cantidad,
    Producto producto) {
40
41         boolean exists = detallePedidos.stream()
42             .anyMatch(detallePedido ->
    detallePedido.getProducto().equals(producto));
43         if (exists) return;
44
45         DetallePedido detallePedido = DetallePedido.
    builder()
46             .cantidad(cantidad)
47             .producto(producto)
48             .subtotal(producto.getPrecio()*
    cantidad)
49             .pedido(this)
50             .build();
51         detallePedidos.add(detallePedido);
52
53     }
54
55     //Buscar detalle por producto
56     public DetallePedido findDetallePedidoByProducto(
    Producto producto) {
57         return detallePedidos.stream()
58             .filter(detallePedido ->
    detallePedido.getProducto().equals(producto))
59             .findFirst()
60             .orElse(null);
61     }
62
63
64     //Eliminar detalle por producto
65     public void deleteDetalleByProducto(Producto
    producto) {
66         detallePedidos.removeIf(d -> d.getProducto().
    equals(producto));
67     }
68
69
70     @Override
71     public void calcularTotal() {
```

```
72         double totalFinal = detallePedidos.stream()
73             .mapToDouble(DetallePedido::
        getSubtotal)
74             .sum();
75         this.total = totalFinal;
76         System.out.println("Total del pedido: " +
        totalFinal);
77     }
78
79
80
81 }
```

```
1 package com.tp.jpamodel.entities;
2 import ar.edu.tup.programacion3.enums.Rol;
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.*;
8
9 @Entity
10 @Table(name = "usuarios")
11 @Getter
12 @Setter
13
14 @ToString(callSuper = true, exclude = "contrasenia")
15 @EqualsAndHashCode(callSuper = false, of = {"mail"})
16
17 @SuperBuilder
18 @NoArgsConstructor
19 @AllArgsConstructor
20 public class Usuario extends Base {
21     private String nombre;
22     private String apellido;
23     @Column(unique = true)
24     private String mail;
25     @Column(unique = true)
26     private String celular;
27     private String contrasenia;
28     @Enumerated(EnumType.STRING)
29     private Rol rol;
30
31     //sets de pedidos
32
33
34     @Builder.Default
35     @OneToMany(mappedBy = "usuario")
36     private Set<Pedido> pedidos = new HashSet<>();
37     public void addPedido(Pedido pedido) {
38         pedidos.add(pedido);
39     }
40
41
```

42

43 }

```
1 package com.tp.jpamodel.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 @Entity
8 @Table(name = "productos")
9 @Getter
10 @Setter
11 @ToString
12 @EqualsAndHashCode(callSuper = false , of = {"nombre"
13     , "precio"})
13 @SuperBuilder
14 @NoArgsConstructor
15 @AllArgsConstructor
16 public class Producto extends Base {
17     @Column(unique = true)
18     private String nombre;
19     private Double precio;
20     private String descripcion;
21     private int stock;
22     private String imagen;
23     private boolean disponible;
24     @ManyToOne
25     @JoinColumn(name = "categoria_id")
26     private Categoria categoria;
27
28
29 }
```

```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import lombok.experimental.SuperBuilder;
6
7 import java.util.HashSet;
8 import java.util.Set;
9
10 @Entity
11 @Table(name = "categorias")
12 @SuperBuilder
13 @Getter
14 @Setter
15 @ToString(callSuper = true, exclude = "productos")
16 @EqualsAndHashCode(callSuper = false, of = {"nombre"
    })
17 @AllArgsConstructor
18 @NoArgsConstructor
19 public class Categoria extends Base {
20     @Column(unique = true)
21     private String nombre;
22     private String descripcion;
23     @OneToMany(mappedBy = "categoria", cascade =
        CascadeType.ALL, orphanRemoval = true)
24     @Builder.Default
25     private Set<Producto> productos = new HashSet
        <>();
26
27     //helper para mantener la bidireccionalidad
28     public void addProducto(Producto producto) {
29         productos.add(producto);
30         producto.setCategoria(this);
31     }
32 }
33
34
35
36
```



```
1 package com.tp.jpa.model.entities;  
2  
3 public interface Calculable{  
4     void calcularTotal();  
5 }  
6  
7
```

```
1 package com.tp.jpa.model.entities;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.JoinColumn;
5 import jakarta.persistence.ManyToOne;
6 import jakarta.persistence.Table;
7 import lombok.*;
8 import lombok.experimental.SuperBuilder;
9
10 @Entity
11 @Table(name = "detalle_pedidos")
12 @Getter
13 @Setter
14 @EqualsAndHashCode(callSuper = false, of = {"producto
    "})
15 @NoArgsConstructor
16 @AllArgsConstructor
17 @SuperBuilder
18 public class DetallePedido extends Base {
19
20     private int cantidad;
21     private Double subtotal;
22     @ManyToOne
23     @JoinColumn(name = "producto_id")
24     private Producto producto;
25     @ManyToOne
26     @JoinColumn(name = "pedido_id", nullable = false)
27     private Pedido pedido;
28
29     public DetallePedido(int cantidad, Double
    subtotal) {
30         this.cantidad = cantidad;
31         this.subtotal = subtotal;
32
33     }
34 }
```